Security Assessment Report: secure-auth-lab

Target	`secure-auth-lab` v1 (Flask + SQLite authentication service), commit `4e665d7`
Assessor / owner	_____ (replace with your name)
Date	_____
Type	Authorized white-box assessment of my own local application, with remediation and retest
Scope & rules	`assessment/SCOPE.md`
Data handling	Sanitized: synthetic accounts and passphrases only, RFC 5737 documentation IPs, no real credentials or personal data

1. Executive summary

I assessed the authentication service I built for the Secure Authentication Lab across five areas: configuration, authentication, validation, dependencies and logging. The core controls held up: no plaintext passwords, generic login errors, equal response time for real and unknown accounts, sound session handling, and clean rejection of hostile input. The assessment still found **five real weaknesses** (three Medium, two Low) and verified two accepted design trade-offs.

All five findings were fixed in separate commits, each with a regression test that **fails on the original code and passes after the fix** (13 of the 16 new tests fail on the baseline; the other 3 guard unchanged behavior). The retest of the same 17 probes went from **6 FAIL / 8 PASS** to **0 FAIL / 14 PASS**, with 2 accepted risks and 1 check (a dependency vulnerability scan) that could not run offline and is wired into CI instead.

ID	Finding	Severity	Status	Fix commit
F1	Database file holding all password hashes was world-readable (`0644`)	Medium	Fixed and retested	`5cce77d`
F2	scrypt cost below OWASP guidance; stored parameters not bounded	Medium	Fixed and retested	`51c8c47`
F3	No security audit logging	Medium	Fixed and retested	`634c1c1`
F4	Rate-limiter state never reclaimed (unbounded memory growth)	Low	Fixed and retested	`47d876d`
F5	Unpinned dependencies; no vulnerability scan in the workflow	Low	Fixed (scan runs in CI)	`217827c`
R1	A third party can lock out a known account for 15 minutes	Low	Accepted risk	n/a

ID	Finding	Severity	Status	Fix commit
R2	Registration reveals whether a username exists	Low	Accepted risk	n/a

Severity = likelihood x impact in the stated threat model: a small service that could be exposed behind HTTPS, where local users and the open internet are both plausible attackers.

2. Scope, authorization and method

I own the application and the machine it runs on and authorized myself to test it (statement and rules of engagement in `assessment/SCOPE.md`). Testing was white-box (source review) plus 17 reproducible probes (`assessment/probes.py`) that drive the app in-process through Flask's test client. No network sockets were opened, each probe used a throwaway database in a temp directory, and request volumes stayed in the hundreds.

Areas tested: **configuration** (C01-C03), **authentication** (C04-C08, C15-C17), **validation** (C09-C10), **logging** (C11), **resource use** (C12), **dependencies** (C13-C14).

Reproduce:

```
git checkout 9da86d9 && python assessment/probes.py   # baseline: 6 FAIL
git checkout main    && python assessment/probes.py   # retest:   0 FAIL
```

Raw output is committed in `evidence/` (`baseline_results.*`, `retest_results.*`, `retest_unit_tests.txt`, `regression_tests_on_baseline.txt`).

3. Findings

F1: Database file readable by other local users (Medium)

- **What:** SQLite created `auth.db` with the process default umask, giving mode `0644`. Any local user could copy the file and attack every stored password hash offline.
- **Evidence:** probe C03 at baseline: `file mode created by create_app under umask 022: 0o644`.
- **Reproduce:** `python assessment/probes.py | grep C03`
- **Impact:** confidentiality of all password hashes against any local account; this compounds F2 because the hashes were also cheaper to crack than recommended.
- **Fix:** `_secure_db_file()` creates the file with `0600` and tightens an existing file at startup.
- **Retest:** C03 PASS (`0o600`). Regression tests `TestF1` (new file under umask `000`, existing `0644` file tightened, app still works).

F2: scrypt cost below OWASP guidance; stored parameters unbounded (Medium)

- **What:** hashes used N=2^14, r=8, p=1. OWASP's equal-strength scrypt options start at N=2^17/p=1, N=2^16/p=2 or N=2^15/p=3, so v1 was a few times too cheap for an attacker to crack. Separately, `verify_password` honoured whatever parameters a database row contained (up to about 512 MiB and 1.7 s per attempt), so a tampered row could force expensive work.
- **Evidence:** C05 at baseline: `stored cost: N=2^14, r=8, p=1`. C17 at baseline: `verify() of a hash with`

`N=2^19 (512 MiB) returned False after 1.73 s`.

- **Impact:** faster offline cracking after any hash leak; the parameter issue needs database write access, so it is defense in depth, not independently rated.
- **Fix:** N=2^15, r=8, p=3 (32 MiB, about 0.25 s per hash) and upper bounds on stored parameters (N up to 2^17, r and p up to 16). Existing v1 hashes still verify and are **upgraded transparently at the next successful login** via `needs_rehash`.
- **Retest:** C05 PASS (`N=2^15, r=8, p=3`), C17 PASS (rejected in 0.00 s). Timing check C07 still PASS (ratio 1.03). Regression tests `TestF2` include the legacy-hash upgrade path.
- **Trade-off:** each login now costs about 0.25 s of CPU and 32 MiB of RAM. Throttling only counts failures, so a flood of valid logins is bounded by the host, not the limiter. Noted as residual risk.

F3: No security audit logging (Medium)

- **What:** the service wrote nothing about failed logins, lockouts, successful logins or logouts.
- **Evidence:** C11 at baseline: `0 audit records captured; missing event/result pairs: [logout/success, password_login/failure, ..., register/success]`.
- **Impact:** brute force, spraying or an account takeover would be invisible, the exact patterns found in the Security Log Investigation task. Detection and incident response would be impossible.
- **Fix:** `auth_lab/audit.py` emits one JSON line per event (`register`, `password_login` with `success`/`failure`/`throttled`, `account_throttle`, `logout`) with `ts`, `event`, `result`, `user_id`, `source_ip` and an internal `reason`. Field names match the investigation log, so the same R1-R3 rules apply. Passwords, hashes and tokens are never logged; JSON encoding neutralizes log-injection attempts (a username containing a newline stays one record); attacker-controlled values are truncated to 64 characters; the optional file sink is `0600`.
- **Retest:** C11 PASS (`11 audit records captured; missing pairs: none; secrets leaked: False`). Regression tests `TestF3`: expected events and fields, no secrets, log injection, file permissions.
- **Note:** the internal `reason` field distinguishes `unknown_user` from `bad_password`. Clients still only ever see the generic message, but the log file must stay access-controlled (it is created `0600`).

F4: Rate-limiter memory never reclaimed (Low)

- **What:** limiter entries were pruned only when the same key was seen again, so each unique username or IP stayed in memory forever.
- **Evidence:** C12 at baseline: `limiter keys held: 330 after 300 unique failures; 332 after the window expired and one more request`.
- **Impact:** slow memory growth an attacker can drive by submitting unique usernames from many IPs (a Low-impact denial of service; per-IP limits bound the rate).
- **Fix:** a time-based sweep every `window/2` removes only fully expired keys. A hard cap with eviction was deliberately **not** used, because evicting live keys would let an attacker flush a victim's lockout.
- **Retest:** C12 PASS (`330 keys held; 2 after the window expired`). Regression tests `TestF4` include "a sweep never removes an active lockout".

F5: Unpinned dependencies and no vulnerability scan (Low)

- **What:** `requirements.txt` used `Flask>=3.0` and listed `pytest` although the tests use `unittest`. Builds were not reproducible and a compromised or vulnerable release could be pulled silently.
- **Evidence:** C13 at baseline: `unpinned: ['Flask>=3.0', 'pytest>=8.0']`.
- **Fix:** exact pins for Flask 3.1.3 and its full dependency tree as tested; CI workflow runs the unit tests, the

probes (failing the build on any FAIL) and `pip-audit`; weekly Dependabot updates.

- **Retest:** C13 PASS. **Limitation:** the vulnerability scan itself (C14) was **not executed** because the assessment sandbox has no network. It is configured in CI and must be run once before relying on the pins.

4. Controls verified (no finding)

Probe	Result
C01 Debug and testing modes off by default	PASS
C02 Cookie flags (`__Host-`, `HttpOnly`, `Secure`, `SameSite=Lax`, no `Domain`) and security headers	PASS
C04 No plaintext passwords; scrypt storage	PASS
C06 Identical generic login errors; throttling does not reveal which accounts exist	PASS
C07 Timing: known vs unknown account (ratio 1.03, n=8; small sample, so indicative only)	PASS
C08 Session rotation, server-side logout, idle expiry, no fixation, tokens hashed at rest	PASS
C09 11 hostile or malformed payloads (type confusion, SQL metacharacters, NUL bytes, oversized, wrong content type)	PASS: 4xx JSON, no 5xx
C10 Deeply nested JSON inside the body cap	PASS

Hypotheses that did not hold up. I expected deeply nested JSON (C10) to trigger an unhandled `RecursionError` and an HTTP 500. It returned a clean 400, so no finding was raised. The first run of the generic-error probe (C06) also failed, but the cause was my probe, not the app: the real account already had one recorded failure before the comparison began. I fixed the probe to equalize starting state and the check passed on unchanged code. Finally, my first oversized-parameter probe (N=2^20) was rejected only because of an incidental `hashlib` memory limit, so I re-aimed it at the largest value the code actually accepts (N=2^19), which is what C17 measures.

5. Accepted risks and residual risk

- **R1: Targeted lockout (C15).** Five wrong guesses from one IP lock a known account for 15 minutes, including against the real user at another IP. Evidence: HTTP 429 for the correct password. Accepted because loosening it would make distributed brute force easier; mitigations are the short lock, `Retry-After`, and the new `account_throttle` audit event for alerting.
- **R2: Username enumeration at registration (C16).** A duplicate registration returns 409. Accepted for a lab service; the real fix is email-based sign-up, which is out of scope.
- **Other limitations:** in-memory limiter (resets on restart, not shared across processes); no MFA, password reset or breached-password check; client IP uses `remote_addr`, so behind a reverse proxy trusted-proxy handling must be configured; valid-login floods are not throttled (relevant now that each hash costs more).

6. Limitations of this assessment

White-box, single assessor, local in-process testing only. No network-level testing, fuzzing campaign, or review of Flask/Werkzeug internals. Probe thresholds (for example the timing ratio) are judgments, and the timing sample is small. The dependency vulnerability scan did not run. Absence of findings is not proof of absence of vulnerabilities.

7. Sanitization statement

This repository and report contain no real credentials, tokens, personal data, or third-party system details. All usernames and passphrases are fictional test values; all IP addresses are RFC 5737 documentation ranges; evidence files were generated from throwaway databases that were deleted after each probe.

8. Commit history (remediation)

Commit	Purpose
`4e665d7`	Baseline: the app as submitted for the Secure Authentication Lab
`9da86d9`	Scope/ROE, probes, baseline results
`5cce77d`	Fix F1: database file permissions
`51c8c47`	Fix F2: scrypt cost and parameter bounds
`634c1c1`	Fix F3: audit logging
`47d876d`	Fix F4: limiter memory reclamation
`217827c`	Fix F5: dependency pins and CI